Easy Campus Stay Assignment 2 Windows video recording kit

Group 22 | Teams recording | Target 7 minutes 48 seconds | Simple English

Mujaffar operates one shared Windows desktop throughout. Ramyavarshini, Yash and Ambuj explain their assigned sections using their real voices. This kit tells each person what to show, what to run and what to say.

Assignment 2 focuses on Docker deployment. Demonstrate build/start, running containers, a successful booking, a genuine deployment problem and its fix, and restart. Add the Minikube section only when it has actually worked.

The institution recommends approximately 6-8 minutes. The schedule includes an optional Minikube extension. Builds and cluster startup may take longer than their speaking slots. Practise first and pre-download images; still show the real commands and results.

If the instructor permits edited waits, mark each cut as a wait removed and retain the original evidence. If editing is not permitted, use the core-only schedule on the next page rather than speeding up or inventing results.

Read the speech paragraphs naturally. Commands are actions for Mujaffar, not sentences to read aloud. Describe a PASS or confirmed booking only after the actual result appears.

Required companion: the updated Windows installation guide. Resolve the missing deploy/configure.py file and finish a complete practice before recording.

1 Running order and who operates

Time	Speaker	Action
0:00-0:35	Mujaffar	Team and project overview
0:35-1:00	Ramyavarshini	Container architecture
1:00-1:30	Yash	Separate Docker images
1:30-2:00	Ambuj	Readiness and database volume
2:00-2:40	Mujaffar	Actual build and startup
2:40-3:00	Ramyavarshini	Containers and API health
3:00-3:35	Mujaffar	Normal booking
3:35-4:15	Yash	Actual symptom and diagnosis
4:15-4:40	Ambuj	Cause and relevant configuration
4:40-5:10	Mujaffar	Actual fix and retest
5:10-5:35	Ambuj	Restart and earlier booking
5:35-6:00	Ramyavarshini	Optional Minikube resources
6:00-6:30	Ramyavarshini	Three replicas and replacement
6:30-7:00	Yash	Booking through Kubernetes
7:00-7:48	All four	12 seconds each for learning

Mujaffar clicks, types and changes windows for every row. No keyboard or screen-sharing handover is needed. The named person explains the actual result on his screen.

Core-only alternative

If optional Minikube is not being submitted, replace 5:35-7:00 with: 5:35-6:00 Ramyavarshini shows saved Compose evidence; 6:00-6:25 Yash explains the actual incident learning; 6:25-6:50 Ambuj shows a notification retry; 6:50-7:38 all four give 12-second closing lines. Keep all core demonstrations.

The timed pages below use the extension schedule. For core-only recording, use the same first ten numbered sections and the alternative in section 15.

2 Prepare Windows and Teams before recording

1. Finish a complete Windows Compose practice and save the real incident evidence. If presenting Minikube, practise that too. Keep a valid private .env in the checkout.
2. Docker Desktop must show a running Linux engine. Stop Assignment 1 launchers. Do not run scripts/run.py for this video.
3. Open Terminal A and Terminal B at C:\Projects\campusstay_assignment2. Keep the browser at http://127.0.0.1:8180/ and an editor with the source files in section 16. Open the evidence folder and architecture diagram.

```
Set-Location 'C:\Projects\campusstay_assignment2'
docker compose config --quiet
```

4. Pre-build and practise so Internet downloads do not consume the recording. Before the recorded build/start step, leave Compose stopped with docker compose down. Retain the database volume and private configuration.
5. All four join the same Teams meeting using headphones. Mujaffar shares the desktop containing the browser, terminal and editor, and keeps sharing throughout. Close private windows first.
6. Agree who can record under your Teams account policy. Make a short rehearsal: all four say one line while Mujaffar switches browser, terminal and editor. Play it back to confirm readable text and all voices.
7. Increase terminal font size and browser zoom. Pre-open the relevant functions and YAML sections. Keep .env and Secret data off screen.

Terminal A holds the foreground Compose output. Terminal B runs diagnostics and checks. Terminal C is used later for the Minikube UI port-forward. Run commands only in the terminal named on each page.

Have a shared timer. The next speaker unmutes and gives a short cue such as Mujaffar, please show Terminal B. Mujaffar performs the action, then the speaker explains it.

3 Introduction from 0 to 1 minute

0:00-0:35 Mujaffar

Show: team slide or report architecture page with Easy Campus Stay, Group 22 and all four names. Commands: none.

Mujaffar says

"Hello, we are Group 22. Our project is Easy Campus Stay, a simple room-booking system for campus visitors. It has Inventory, Reservation, Payment and Notification. Users can check rooms, make a booking and receive a demo confirmation. In Assignment 2, we run the same application in Docker. I am Mujaffar. I handle Reservation and the browser interface."

0:35-1:00 Ramyavarshini

Show: architecture diagram. Point to the two Python APIs, the PHP mocks, browser interface and four logical databases.

Ramyavarshini says

"I am Ramyavarshini. I handle Inventory. Inventory and Reservation use Python. Payment, Notification and the browser interface use PHP. Each service has its own database and login. One MariaDB container stores these four databases. The services communicate through REST APIs."

If optional Kubernetes is included, Mujaffar can add: We will also show a booking through Kubernetes and recovery after deleting one Inventory pod. Say this only when that section will be shown.

4 Dockerfiles from 1 minute to 1 minute 30

Speaker: Yash. Operator: Mujaffar. Show the two files in the editor. These commands open files; they do not start services.

```
notepad services\inventory\Dockerfile
notepad services\payment\Dockerfile
```

1. In Inventory Dockerfile, point to FROM python:3.12-slim-bookworm, the requirements copy/install, USER app and CMD.
2. Switch to Payment Dockerfile. Point to FROM php:8.3-apache-bookworm, PDO MySQL extension, public/source copies and apache2-foreground.

Yash says

"I am Yash. I handle the Payment mock. This Dockerfile gives Inventory its Python runtime and packages. It copies the API code and starts the service as an application user. Payment uses PHP with Apache and the MySQL extension. Each service has its own image. Passwords are supplied when the containers run."

Do not explain every Dockerfile line. The important ideas are separate images, correct runtime, copied source, startup command and configuration outside the image. The existing business rules remain in the service source folders.

5 Compose from 1 minute 30 to 2 minutes

Speaker: Ambuj. Operator: Mujaffar. Open docker-compose.yml and use editor search to locate each named section.

```
notepad docker-compose.yml
```

1. Show db healthcheck, db-init depends_on with service_healthy, and an application dependency on service_completed_successfully.
2. Show reservation environment INVENTORY_URL set to http://inventory:8080 and the database_data named volume.

Ambuj says

"I am Ambuj. I handle the Notification mock. Compose starts the complete application. First the database becomes ready. Then db-init creates the databases and service users. After that, the APIs and browser interface start. Containers use service names to find each other. This named volume keeps the database data when containers are recreated."

The YAML contains password variable references, not actual passwords. Keep the private .env closed. Show that the database has no published Windows port.

Code path: docker-compose.yml. This is configuration to explain; run the startup command on the next page.

6 Build and health from 2 to 3 minutes

2:00-2:40 Mujaffar uses Terminal A

```
Set-Location 'C:\Projects\campusstay_assignment2'
docker compose config --quiet
docker compose up --build
```

Mujaffar says

"We build and start the complete application with one command. Docker prepares the separate images. Compose creates the containers and connects them. This is the actual output from our computer. We wait until the services are healthy."

Leave Terminal A running. If the build needs longer, wait. The slot is a rehearsal target, not a reason to claim startup early.

2:40-3:00 Ramyavarshini uses Terminal B

```
Set-Location 'C:\Projects\campusstay_assignment2'
docker compose ps -a
Invoke-RestMethod http://127.0.0.1:8101/health
Invoke-RestMethod http://127.0.0.1:8102/health
```

Ramyavarshini says

"The database, four APIs and browser interface are running. db-init exited with code zero because its setup work is complete. These API health responses show access to each service database. We will now check the full booking workflow."

Expected: Six containers stay running and healthy. db-init Exited (0) is expected. The four API health cards can also be shown in Service tests in the browser.

7 Booking from 3 minutes to 3 minutes 35

Speaker and operator: Mujaffar. Show http://127.0.0.1:8180/. Commands: none.

1. Open Service tests, refresh the service cards and click Reset to normal. Return to Book a room.
2. Keep a future date. Click Add sample rooms ONCE. Point to five of five rooms and INR 2500.
3. Keep a synthetic guest and guest@example.test. Click Confirm booking ONCE and wait.
4. Point to CONFIRMED, full booking ID, successful payment/notification and four of five rooms remaining. Keep this booking ID for the restart section.

Mujaffar says

"This browser is using our Docker deployment. I add five sample rooms and book one room. Reservation asks Inventory to hold a room, then calls Payment and Notification. The booking is confirmed and four rooms remain. These are demo payment and notification services. No real money or email is sent."

Say the result only when it appears. Refresh this booking checks the same request; Confirm booking normally creates another one.

8 Actual incident from 3 minutes 35 to 4 minutes 40

3:35-4:15 Yash explains symptom and diagnosis

Show the original observed error and its real timestamp, then the targeted diagnostic output. Fill docs/TROUBLESHOOTING_WORKSHEET.md before the final take.

```
docker compose ps -a
docker compose logs --tail=80 reservation
```

Yash says

"During our deployment, we observed [actual symptom]. We checked the container status and these logs. The output showed [actual error]. We then checked [the relevant setting or dependency] to find the cause."

Replace every bracket with the real observation. Do not read brackets in the video. For a database or image failure, use that service log instead of Reservation.

4:15-4:40 Ambuj explains the cause

Ambuj says

"The cause was [actual cause]. This setting matters because [simple explanation]. The log and this configuration value support our diagnosis. We will now correct it and repeat the failed action."

Only for an actual wrong Inventory URL

```
docker compose exec reservation python -c `
  "import os; print(os.environ['INVENTORY_URL'])"
docker compose exec reservation python -c `
  "import socket; print(socket.gethostbyname('inventory'))"
```

Simple explanation for that specific cause: localhost inside Reservation refers to Reservation itself. Inventory is reached using http://inventory:8080. Use this explanation only if it matches the real incident.

The missing deploy/configure.py setup error happened before Docker ran. It alone does not meet the container deployment incident requirement. A deliberately injected localhost fault must be labelled as injected.

9 Fix and validation

4:40-5:10 Mujaffar

Speaker and operator: Mujaffar. Show the exact actual change, then the affected service and failed workflow again.

For an environment setting corrected in the base Compose configuration, recreate only the affected service. This example is for Reservation:

```
docker compose up -d --no-deps --force-recreate reservation
docker compose logs --tail=80 reservation
```

For a source or Dockerfile change, rebuild the affected image instead. Example:

```
docker compose up -d --build --no-deps reservation
```

Mujaffar says

"We changed [actual setting or code] and recreated the affected container. We now repeat the failed action. This result shows that [actual workflow] works after the fix."

1. For a recoverable pending booking, refresh that same booking. If it was already cancelled, make a new booking and explain that clearly.

Expected: Only say CONFIRMED after the booking actually reaches that state. A running container by itself is not enough validation.

If practising the supplied override, remove it by using the base Compose recreation command above. Say: We deliberately introduced this configuration fault for this exercise. Do not present a rehearsal as an unplanned incident.

10 Restart from 5 minutes 10 to 5 minutes 35

Speaker: Ambuj. Operator: Mujaffar. In Terminal B:

```
docker compose down
docker compose up -d
docker compose ps -a
```

1. Wait for health. In the browser, use Already have a booking ID? and enter the earlier normal booking ID. Show its confirmed status and the same room/date stock.

Ambuj says

"We stopped and recreated the containers. We kept the named database volume. This is the booking we made before the restart. It is still confirmed, so the data survived container recreation."

Do not add -v. Keep the same private configuration. A new booking after restart alone does not prove that the earlier data survived.

If restart needs longer, wait for the actual health state. For the optional route, save Compose evidence before switching to Minikube.

11 Optional Minikube from 5 minutes 35 to 6 minutes

Speaker: Ramyavarshini. Operator: Mujaffar. Do this section only after practising the optional deployment. The speaking slot shows the real completed startup and resources; do not assume a cold cluster can start in 25 seconds.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Capture-Compose.ps1
docker compose down
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Deploy-Minikube.ps1
kubectl --context=minikube -n easy-campus-stay get pods,jobs,services,pvc
```

Ramyavarshini says

"We also deploy the same images to Minikube, a local Kubernetes cluster. Deployments manage the application pods. Services provide stable names. The database has persistent storage, and the setup Job has completed. We must also show that a booking works through this deployment."

1. Briefly show k8s/30-application.yml at Inventory replicas and readinessProbe. A ready pod can receive traffic; the health check includes its database access.

2. Open Terminal C and leave this command running:

```
kubectl --context=minikube -n easy-campus-stay `
  port-forward service/ui 8180:8080
```

Compose stays stopped. The browser on port 8180 now reaches the Kubernetes UI Service through this port-forward. Compose and Minikube have different database data.

12 Scaling from 6 minutes to 6 minutes 30

Speaker: Ramyavarshini. Operator: Mujaffar. In Terminal B:

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Demonstrate-SelfHealing.ps1
kubectl --context=minikube -n easy-campus-stay get pods -l app=inventory
```

Ramyavarshini says

"We ask Kubernetes to keep three Inventory replicas. We delete one pod. The Deployment creates a replacement because the required count is still three. The deleted pod UID is gone, and three replicas are ready again. This shows recovery after deleting a pod."

1. Show evidence\assignment2\minikube\self-healing.json. Point to deletedUid, readyReplicas 3 and result PASS only after the helper succeeds.

Relevant code: deploy/windows/Demonstrate-SelfHealing.ps1. Find scale deployment/inventory, delete pod and the final ready-replica/UID check.

Simple code explanation if asked: This script saves the pod identities before deletion, removes one Inventory pod, and checks that three ready replicas exist afterward. A replacement must have a new UID.

This is manual scaling and Deployment recovery. It is not automatic scaling based on load.

13 Kubernetes booking

6:30-7:00 Yash

Speaker: Yash. Operator: Mujaffar. Terminal C keeps port forwarding. Show the browser at http://127.0.0.1:8180/.

1. Create NEW sample rooms in Kubernetes and make a normal booking. Show CONFIRMED, successful payment/notification and four of five remaining.

Yash says

"This booking goes through the Kubernetes UI Service. It confirms that the APIs and database work together after Inventory pod replacement. We also run the automated checks and save their real outputs."

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Check-Minikube.ps1
```

2. Show the actual successful console result and evidence\assignment2\minikube\workflow-console.txt. If it is still running, wait before describing it as passed.

The booking is the essential advanced evidence. Running pods alone are insufficient. The original Linux reference evidence is not a substitute for these cluster outputs.

14 Closing from 7 minutes to 7 minutes 48

Show the safe evidence folder and the final repository/video link slide. Each member has about 12 seconds. Adapt each statement to their actual work.

Mujaffar says

"I learned how service configuration changes inside containers. I reviewed Reservation and the browser connection and checked the booking workflow."

Ramyavarshini says

"I learned to check both pod readiness and application behaviour. I reviewed Inventory and checked availability after deployment and recovery."

Yash says

"I learned to use logs and service addresses to find deployment problems. I reviewed Payment and checked it through a complete booking."

Ambuj says

"I learned how health checks and persistent storage support deployment. Payment and Notification are mocks. We reviewed the AI-assisted files and saved our actual evidence. Thank you."

Do not claim a review or test that a person has not done. If the optional task was omitted, replace recovery references with the actual Compose checks.

1. Mujaffar stops recording after the closing. Play back the final file. Check readable text, all four real voices, correct links and actual results. Save the video URL in the report.

After recording: stop Terminal C with Ctrl+C and use minikube stop if used. For Compose, use docker compose down while retaining its database volume.

15 Core only continuation when Minikube is omitted

Use this page instead of the optional timed pages. Keep the same first 5 minutes 35 seconds, then follow these steps. Total target: 7 minutes 38 seconds.

5:35-6:00 Ramyavarshini shows saved Compose evidence

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Capture-Compose.ps1
explorer evidence\assignment2\compose
```

Ramyavarshini says

"These files record our Docker versions, container states, image identities and logs. Our workflow summary contains the actual checks. We keep reference evidence separate from our Windows deployment results."

6:00-6:25 Yash explains the real incident learning

Yash says

"Our deployment problem was [actual problem]. The useful diagnostic command was [actual command]. We fixed [actual cause] and repeated the booking. We would add [actual preventive check] before the next deployment."

6:25-6:50 Ambuj shows notification retry

In Service tests choose Successful payment and Fail once, then retry. Go to booking and make one new booking. Inspect notification.

Ambuj says

"This is a new booking with a notification that fails once. The booking remains confirmed. The retry sends the demo notification, and the attempts count is two."

6:50-7:38 all four close

Use the four 12-second closing lines on the preceding page, adjusted to the work actually performed. State that optional Kubernetes work is not being claimed.

16 Code paths and simple explanations

File and what to find	Simple explanation
services/inventory/Dockerfile FROM, requirements, USER app, CMD	This image contains the Python API runtime and starts Inventory.
services/payment/Dockerfile FROM, pdo_mysql, Apache startup	This image runs the PHP mock with Apache and database support.
docker-compose.yml db, db-init, depends_on, reservation URLs, database_data	This file starts the deployment, waits for dependencies and retains database data.
ui/bootstrap.php ui_settings and upstream	The UI reads configured API URLs and allows the approved service host names.
deploy/provision/init.py account grants and schema setup	Setup creates four separate service databases and restricted users.
deploy/run_checks.py main and script loop	This runs the original workflows through container APIs and checks database isolation.
k8s/30-application.yml Inventory Deployment, replicas, readinessProbe	Kubernetes maintains the requested application pods and checks readiness.
deploy/windows/Demonstrate-SelfHealing.ps1 scale, delete and UID checks	This script proves replacement after deleting one Inventory pod.

These files are to display and explain, not code fragments to paste into PowerShell. Open them in an editor and use Ctrl+F to find the named item.

The four service repository files still contain the Assignment 1 business rules. A long repeat of concurrency/idempotency/saga code would use time needed for Assignment 2 deployment evidence.

17 Final recording and submission checks

1. The video shows a real build/start command, actual running services and at least one successful booking through Compose.
2. The genuine deployment incident has the observed symptom, diagnostic evidence, actual fix, successful retest and a clear learning. Replace all bracketed speaking prompts before recording.
3. The same earlier booking is retrieved after container recreation. The named volume and private configuration are retained.
4. If the optional extension is claimed, show Minikube resources, the working Kubernetes booking, three Inventory replicas, the deleted/replacement pod evidence and a working flow afterward.
5. All four use their real voices and can explain the files they reviewed. Mujaffar shares and operates the same desktop throughout.
6. No private .env, passwords, tokens, Secret contents or resolved environment dumps appear on screen. Review any logs before publishing them.
7. The report has no more than five main pages excluding appendices. Add actual versions/results, member contributions, real Assignment 2 commit references, AI disclosure and accessible links.
8. Keep original evidence when any permitted long waits are cut. Label the cuts. Do not replace command results with invented PASS screens.

The kit provides speech for a genuine demonstration; it does not establish that the Windows Docker or Kubernetes run has already succeeded. Complete the installation guide and practice first.